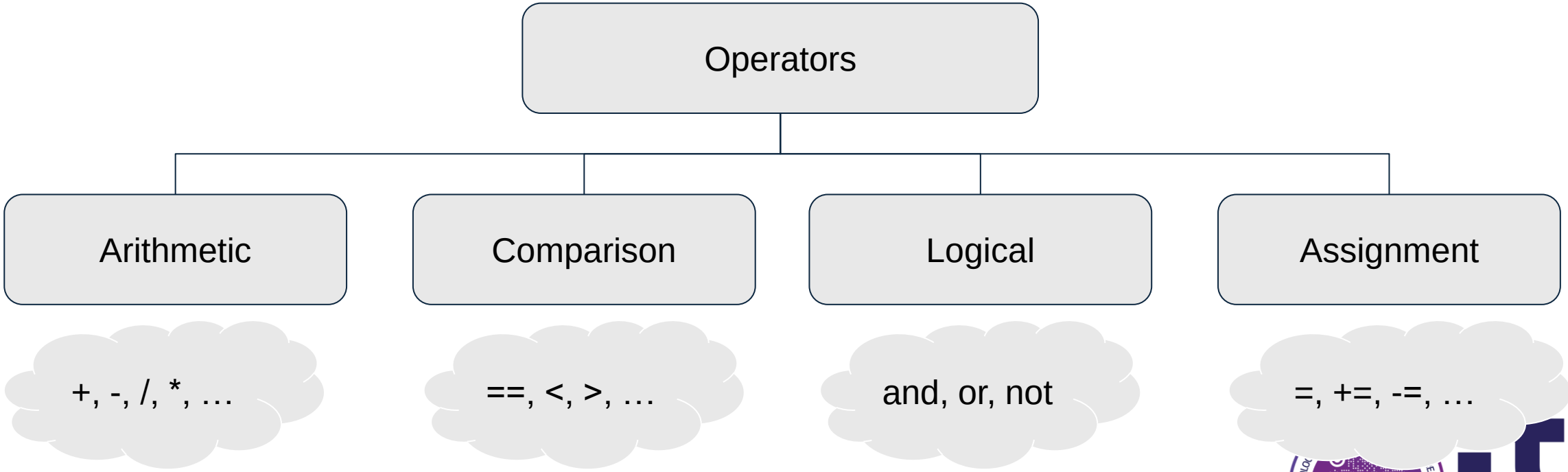
4. Operators

Modifying data

What are operators?

Operators are used to **perform operations** on variables and values.

Python supports **various types** of operations, including arithmetic, comparison, logical, and assignment operations.



ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Arithmetic Operators

Addition	+
Subtraction	-
Multiplication	*
Division	/
Exponentiation	**
Floor division	//
Modulo	%

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Arithmetic Operators

Example:

```
x = 10

y = 5

print(x + y)

print(x - y)

print(x * y)

print(x / y)

print(x ** y)
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Arithmetic Operators

Example:

```
x = 10

y = 5

print(x + y)            -> 15

print(x - y)            -> 5

print(x * y)            -> 50

print(x / y)            -> 2.0

print(x ** y)           -> 100000
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Arithmetic Operators

Example:

```
x = 10

y = 5

print(y / x)

print(y // x)

print(x%y)
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Arithmetic Operators

Example:

```
x = 10

y = 5

print(y / x)                -> 0.5

print(y // x)               -> 0

print(x%y)                  -> 0
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Comparison Operators

equal to	==
not equal to	!=
greater than	>
less than	<
greater than or equal to	>=
less than or equal to	<=

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Comparison Operators

Example:

```
x = 10

y = 5

print(x == y)

print(x != y)

print(x > y)

print(x < y)

print(x >= y)

print(x <= y)
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Comparison Operators

Example:

```
x = 10

y = 5

print(x == y)     -> False

print(x != y)     -> True

print(x > y)      -> True

print(x < y)      -> False

print(x >= y)     -> True

print(x <= y)     -> False
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Logical Operations

`and`	Returns True if **both** the operands are True
`or`	Returns True if **either** of the operands is True
`not`	Returns True if the operand is False

X	Y	X and Y	X or Y	not(X)	not(Y)
T	T	T	T	F	F
T	F	F	T	F	T
F	T	F	T	T	F
F	F	F	F	T	T

its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Logical Operators

Example:

```
x = True

y = False

print(x and y)

print(x or y)

print(not x)

print(x and not y)
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Logical Operators

Example:

```
x = True

y = False

print(x and y)    -> False

print(x or y)     -> True

print(not x)      -> False

print(x and not y) -> True
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Assignment Operators

Operator	Example	Same As
=	`x = 1`	`x = 1`
+=	`x += 1`	`x = x + 1`
-=	`x -= 1`	`x = x - 1`
*=	`x *= 1`	`x = x * 1`
/=	`x /= 1`	`x = x / 1`
//=	`x //= 1`	`x = x // 1`
%=	`x %= 1`	`x = x % 1`
**=	`x **= 1`	`x = x ** 1`

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Assignment Operators

Example: What's the value of x after every line? Discuss with your seatmate!

```
x = 10

x += 5

x -= 3

x *= 2

x /= 4

x //= 2

x %= 2

x **= 3
```

its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Assignment Operators

Example:

```
x = 10          -> x = x + 5        -> x = 15
x += 5          -> x = x - 3        -> x = 12
x -= 3          -> x = x * 2        -> x = 24
x *= 2          -> x = x / 4        -> x = 6.0
x /= 4          -> x = x // 2       -> x = 3.0
x //= 2         -> x = x % 2        -> x = 1.0
x %= 2          -> x = x ** 3       -> x = 1.0
x **= 3
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Operator Precedence

Operator precedence in Python refers to the rules that **determine** the **order** in which operators are **evaluated** in an *expression*. When an expression contains multiple operators, Python follows a specific *order of precedence* to determine which operations are performed first.

2 + 3 * 4

2 + 3**2 / 3

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Operator Precedence

Operator precedence in Python refers to the rules that **determine** the **order** in which operators are **evaluated** in an *expression*. When an expression contains multiple operators, Python follows a specific *order of precedence* to determine which operations are performed first.

2 + 3 * 4

2 + 3**2 / 3

In other words, it follows the **PEMDAS* rule**!

* Parentheses, Exponent, Multiply/Divide, Add/Subtract

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Operator Precedence

`()`	Parentheses
`**`	Exponentiation
`+x  -x  ~x`	Unary plus, unary minus
`*  /  //  %`	Multiplication, division, floor division, and modulus
`+  -`	Addition and subtraction
`==  !=  >  >=  <  <=`	Comparisons, identity, and membership operators
`not`	Logical NOT
`and`	AND
`or`	OR

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Operator Precedence

Example:

```
print(10 - 4 * 2)

print((10 - 4) * 2)
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Operator Precedence

Example:

```
print(10 - 4 * 2)          → 2

print((10 - 4) * 2)        → 12
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Associativity of Python Operators

Associativity is the order in which an expression is evaluated that has **multiple operators** of the **same precedence**. **Almost all** the operators have **left-to-right** associativity.

Example:

```
print(5 * 2 // 3) #read it as (5 * 2) // 3 -> 10 // 3 -> 3

print(5 * (2 // 3)) #using () give precedence to 2 // 3
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Associativity of Python Operators

Associativity is the order in which an expression is evaluated that has multiple operators of the same precedence. **Almost all** the operators have left-to-right associativity.

Example:

```
print(5 * 2 // 3)              → 3

print(5 * (2 // 3))            → 0
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Associativity of Python Operators

Exponent operator ** has right-to-left associativity in Python.

Example:

```
print(2 ** 3 ** 2)     #read it as 2 ** (3 ** 2) -> 2 ** 9 -> 514

print((2 ** 3) ** 2)   #using () give precedence to 2 ** 3
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Associativity of Python Operators

Exponent operator ** has right-to-left associativity in Python.

Example:

```
print(2 ** 3 ** 2)          → 512

print((2 ** 3) ** 2)        → 64
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Associativity of Python Operators

Exponent operator ** has right-to-left associativity in Python.

Example:

```
print(2 ** 3 ** 2)          → 512

print((2 ** 3) ** 2)        → 64
```

NOTE

When in doubt, you can always use () to avoid confusion!

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Other operators

EXOTIC

Identity Operators	`is, is not`
Bitwise Operators	`&, |, ^, ~, <<, >>`
List Operators (will be introduced later)	`in, not in`

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Other operators

Identity Operators Example:

```
a = 5

b = a

c = 7

print(a is b)                    -> True

print(a is c)                    -> False

print(a is not c)                -> True
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Other operators

Bitwise Operators Example:

```
5 & 3 (5 in binary is 101, 3 è 011)   → Result: 1 (001)

5 | 3 (5 in binary is 101, 3 è 011)   → Result: 7 (111)

5 ^ 3 (5 in binary is 101, 3 è 011)   → Result: 6 (110)

~ 5 (5 in binary is 101)              → Result: -6 (~101)

5 << 2 (5 in binary is 101)           → Result: 20 (10100)

5 >> 2 (5 in binary is 101)           → Result: 1 (001)
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Other operators

List Operators Example:

```
string1='hello'


print('h' in string1)                    -> True

print('c' in string1)                    -> False

print('c' not in string1)                -> True
```

String Operators

So far, we've been working with integers, floats, and boolean data.

Some of the operators introduced so far can also work with string data types (but not all of them).

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

String Operators

Name	Operator	Example
Concatenation	s1 + s2	`“My name is “ + “Alice.”`
Repetition	s * n	`“Blah” * 3`
Comparison	s1 == s2	`“Alice” == “Bob”`
	s1 != s2	`“Alice” != “Bob”`

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

String operators

String Operators Example:

```
print("My name is " + "Alice")          -> My name is Alice

print("Blah " * 3)                      -> Blah Blah Blah

print("Alice" == "Bob")                 -> False

print("Alice" != "Bob")                 -> True
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Exercise

Define the following variables:

`x` containing an integer

`y` containing 0

`s` containing a string

Enter the following in your interpreter. What happens? Explain.

```
>>> x / y

>>> s + x

>>> s * y
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

String Formatting

As shown in the last slide, you can't concatenate a string with a number using +

Instead, an easier way is through **string formatting**!

```
name = "Alice"

age = 30

print( f"My name is {name} and I am {age} years old." )


print( f"My name is " + name + " and I am " + age + " years old." )
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

String Formatting

There are various ways to format strings:

```
name = "Alice"

age = 30

print( f"My name is {name} and I am {age} years old." )

print( "My name is {} and I am {} years old.".format(name, age) )

print( f"My name is %s and I am %d years old." % (name, age) )
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

String Formatting

There are various ways to format strings:

```
name = "Alice"

age = 30

print( f"My name is {name} and I am {age} years old." )

print( "My name is {} and I am {} years old.".format(name, age) )

print( f"My name is %s and I am %d years old." % (name, age) )
```

All of these do the same thing!

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

String Formatting: Tips and Tricks

You can limit the decimal places of a numerical value:

```
pi = 3.14159265359

formatted_pi1 = "The value of pi is {:.2f}".format(pi)

formatted_pi2 = f"The value of pi is {pi:.3f}"


print(formatted_pi1) → ???

print(formatted_pi2) → ???
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

String Formatting: Tips and Tricks

You can limit the decimal places of a numerical value:

```
pi = 3.14159265359

formatted_pi1 = "The value of pi is {:.2f}".format(pi)

formatted_pi2 = f"The value of pi is {pi:.3f}"


print(formatted_pi1) → 'The value of pi is 3.14'

print(formatted_pi2) → 'The value of pi is 3.142'
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

String Formatting: Tips and Tricks

You can limit the decimal places of a numerical value:

```
pi = 3.14159265359

formatted_pi1 = "The value of pi is {:.2f}".format(pi)

formatted_pi2 = f"The value of pi is {pi:.3f}"
```

```
print(formatted_pi1) → 'The value of pi is 3.14'

print(formatted_pi2) → 'The value of pi is 3.142'
```

Specifies the number of decimal places

Indicates that it should be formatted as a float

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Escape Characters

Escape characters are special characters in strings that are prefixed with a backslash `\`

Alters the interpretation of the subsequent character in the string.

Common escape characters:

- \n Newline character
- \t Tab character
- \\ Backslash character
- \' Single quote character
- \" Double quote character

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Escape Characters

```
print("Hello\nWorld!")

print("Name:\tAlice")

print("C:\\Users\\Alice\\Documents")

print('It\'s a sunny day.')

print("She said, \"Hello!\".")
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY